

PRD — Don Bosco Connect v2.0

Document de Spécification Complet — Agent-Ready

Version : 2.0

Statut : Prêt pour développement

Audience : Agent IA de développement / Équipe technique

Langue cible : Français (UI) + Arabe RTL (support)

TABLE DES MATIÈRES

- [1. Vision & Contexte](#)
 - [2. Architecture Système](#)
 - [3. Modèle de Données](#)
 - [4. Profils Utilisateurs & Permissions](#)
 - [5. Stack Technique Complète](#)
 - [6. Structure du Projet](#)
 - [7. Configuration Docker Compose](#)
 - [8. Variables d'Environnement](#)
 - [9. Spécification API REST](#)
 - [10. Spécification WebSocket](#)
 - [11. Moteur IA — Spécification Détaillée](#)
 - [12. User Stories & Critères d'Acceptation](#)
 - [13. Interfaces UI par Profil](#)
 - [14. Sécurité & Conformité](#)
 - [15. Contraintes Non-Fonctionnelles \(SLA\)](#)
 - [16. Roadmap MVP Phasée](#)
 - [17. Stratégie de Tests](#)
 - [18. Monitoring & Observabilité](#)
 - [19. Internationalisation](#)
 - [20. KPIs & Métriques de Succès](#)
 - [21. Glossaire](#)
-

1. Vision & Contexte

1.1 Objectif

Créer **Don Bosco Connect**, un écosystème éducatif numérique unifié pour l'école **Don Bosco Tunis** combinant :

- Gestion administrative complète (inscriptions, emplois du temps, notes, absences)
- Collaboration pédagogique enseignant ↔ élève
- Apprentissage adaptatif piloté par l'IA locale (RAG + Ollama)
- Lien école-famille en temps réel

1.2 Périmètre

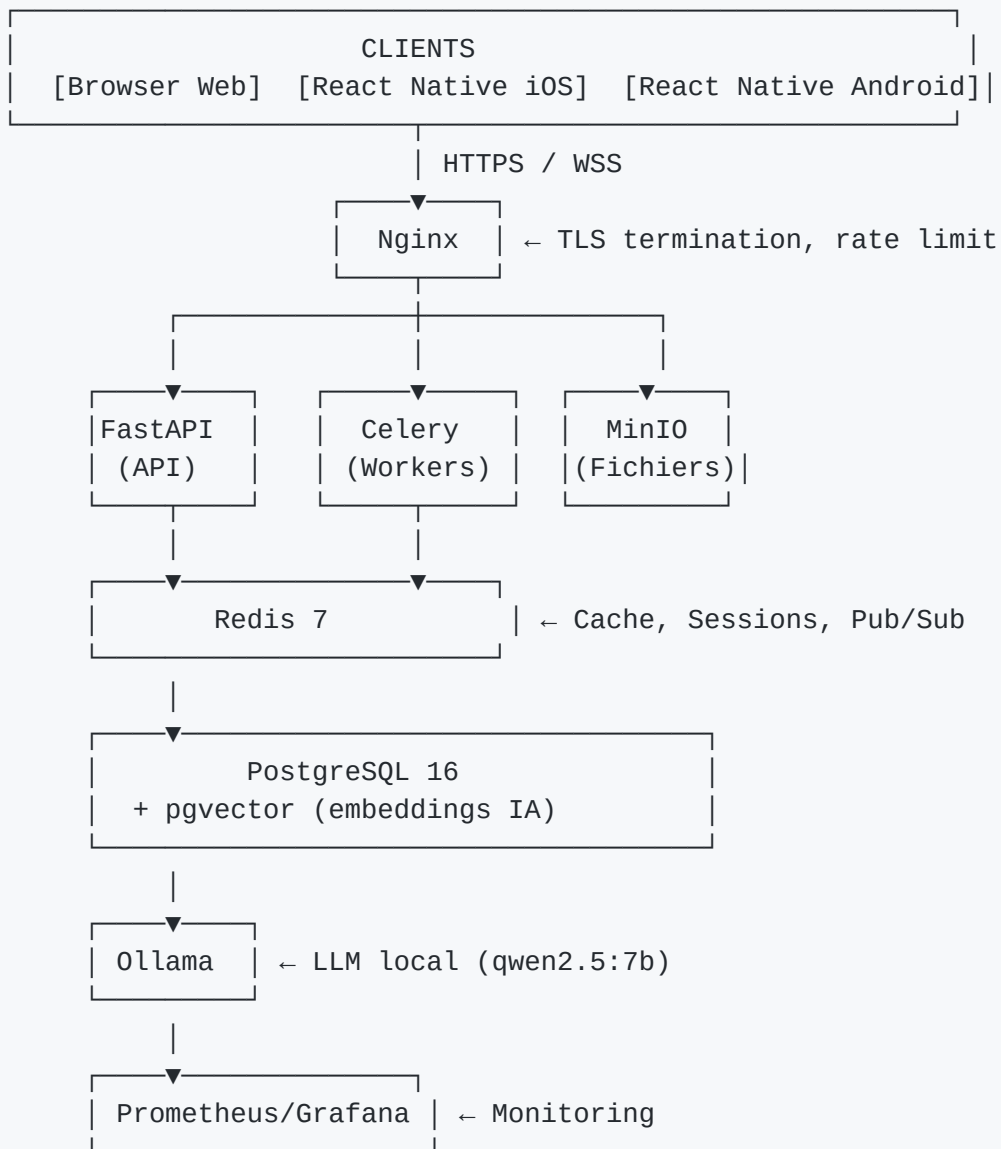
Dans le périmètre	Hors périmètre (v1)
Web app (navigateur)	Application desktop native
App mobile iOS + Android	Intégration MooVle/Moodle
IA locale Ollama	IA cloud (OpenAI, Gemini)
4 profils : Admin, Enseignant, Élève, Parent	Alumni / anciens élèves
Messagerie interne	Email externe
Paieement de scolarité	Comptabilité générale

1.3 Contraintes Clés

- **Tout hébergement local** : aucune donnée ne quitte le réseau de l'école (données de mineurs)
- **Stack 100% open source** : zéro licence propriétaire
- **Connexion minimale** : l'app doit fonctionner sur réseau WiFi scolaire (10 Mbps partagés)
- **Langue principale** : Français ; support RTL arabe obligatoire

2. Architecture Système

2.1 Vue d'ensemble



2.2 Flux de données principaux

Flux 1 — Authentification

Client → POST /auth/login → FastAPI → PostgreSQL (verify hash)
→ Redis (store refresh token) → Client (JWT access + refresh)

Flux 2 — Upload cours + Indexation IA

Enseignant → POST /courses/{id}/files → MinIO (storage)
→ Celery task queued → Redis
→ Worker: PyMuPDF extract → chunk → embed (Ollama nomic-embed-text)
→ pgvector store → Notification WebSocket "cours indexé"

Flux 3 — Mentor IA (RAG)

```
Élève → POST /ai/chat (question) → FastAPI
      → Embed question (Ollama) → pgvector similarity search (top-5 chunks)
      → Build prompt (system + context + question) → Ollama stream
      → SSE token-by-token → Frontend affichage progressif
      → Log conversation → PostgreSQL
```

Flux 4 — Notification absence

```
Enseignant → POST /absences → PostgreSQL
           → Celery task → Redis pub → WebSocket
           → Parent notifié < 5 minutes
```

3. Modèle de Données

3.1 Schéma PostgreSQL complet

```

-- =====
-- EXTENSIONS
-- =====

CREATE EXTENSION IF NOT EXISTS "uuid-ossf";
CREATE EXTENSION IF NOT EXISTS "pgcrypto";
CREATE EXTENSION IF NOT EXISTS "vector";

-- =====
-- AUTHENTICATION & UTILISATEURS
-- =====

CREATE TYPE user_role AS ENUM ('admin', 'teacher', 'student', 'parent');
CREATE TYPE user_status AS ENUM ('active', 'inactive', 'suspended');

CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL, -- bcrypt cost 12
    role user_role NOT NULL,
    status user_status DEFAULT 'active',
    first_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    phone VARCHAR(20),
    avatar_url TEXT,
    preferred_language VARCHAR(5) DEFAULT 'fr', -- 'fr' | 'ar'
    mfa_enabled BOOLEAN DEFAULT FALSE,
    mfa_secret TEXT, -- TOTP secret, encrypted
    last_login_at TIMESTAMPTZ,
    failed_login_count INTEGER DEFAULT 0,
    locked_until TIMESTAMPTZ,
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE refresh_tokens (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID REFERENCES users(id) ON DELETE CASCADE,
    token_hash VARCHAR(255) NOT NULL, -- SHA-256 of token
    device_info TEXT,
    ip_address INET,
    expires_at TIMESTAMPTZ NOT NULL,
    revoked_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE audit_logs (
    id BIGSERIAL PRIMARY KEY,

```

```

    user_id UUID REFERENCES users(id) ON DELETE SET NULL,
    action VARCHAR(100) NOT NULL, -- 'login', 'grade_created', 'file_uploaded', etc.
    resource_type VARCHAR(50),
    resource_id UUID,
    ip_address INET,
    user_agent TEXT,
    metadata JSONB DEFAULT '{}',
    created_at TIMESTAMPTZ DEFAULT NOW()
);

```

```

-- =====
-- STRUCTURE SCOLAIRE
-- =====

```

```

CREATE TABLE academic_years (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    name VARCHAR(50) NOT NULL, -- ex: "2025-2026"
    start_date DATE NOT NULL,
    end_date DATE NOT NULL,
    is_current BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

```

```

CREATE TABLE classes (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    academic_year_id UUID REFERENCES academic_years(id),
    name VARCHAR(50) NOT NULL, -- ex: "3ème A", "Terminale Sciences"
    level VARCHAR(30) NOT NULL, -- ex: "3eme", "terminale"
    section VARCHAR(10), -- ex: "A", "B", "Sciences"
    main_teacher_id UUID REFERENCES users(id),
    max_students INTEGER DEFAULT 30,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

```

```

CREATE TABLE class_enrollments (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    student_id UUID REFERENCES users(id) ON DELETE CASCADE,
    class_id UUID REFERENCES classes(id) ON DELETE CASCADE,
    academic_year_id UUID REFERENCES academic_years(id),
    enrolled_at TIMESTAMPTZ DEFAULT NOW(),
    status VARCHAR(20) DEFAULT 'active', -- 'active', 'transferred', 'graduated'
    UNIQUE(student_id, class_id, academic_year_id)
);

```

```

CREATE TABLE student_parent_links (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    student_id UUID REFERENCES users(id) ON DELETE CASCADE,
    parent_id UUID REFERENCES users(id) ON DELETE CASCADE,

```

```

relationship VARCHAR(30) DEFAULT 'parent', -- 'parent', 'guardian', 'tutor'
is_primary BOOLEAN DEFAULT FALSE,
created_at TIMESTAMPTZ DEFAULT NOW(),
UNIQUE(student_id, parent_id)
);

```

```

-- =====
-- MATIÈRES & EMPLOIS DU TEMPS
-- =====

```

```

CREATE TABLE subjects (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  name VARCHAR(100) NOT NULL,
  name_ar VARCHAR(100), -- nom en arabe
  code VARCHAR(20) UNIQUE NOT NULL, -- ex: "MATH", "FR", "AR"
  color VARCHAR(7) DEFAULT '#3B82F6', -- hex couleur UI
  coefficient DECIMAL(3,1) DEFAULT 1.0,
  description TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

```

```

CREATE TABLE teacher_subject_assignments (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  teacher_id UUID REFERENCES users(id) ON DELETE CASCADE,
  subject_id UUID REFERENCES subjects(id) ON DELETE CASCADE,
  class_id UUID REFERENCES classes(id) ON DELETE CASCADE,
  academic_year_id UUID REFERENCES academic_years(id),
  UNIQUE(teacher_id, subject_id, class_id, academic_year_id)
);

```

```

CREATE TYPE schedule_day AS ENUM
('monday', 'tuesday', 'wednesday', 'thursday', 'friday', 'saturday');

```

```

CREATE TABLE timetable_slots (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  class_id UUID REFERENCES classes(id) ON DELETE CASCADE,
  subject_id UUID REFERENCES subjects(id),
  teacher_id UUID REFERENCES users(id),
  day schedule_day NOT NULL,
  start_time TIME NOT NULL,
  end_time TIME NOT NULL,
  room VARCHAR(50),
  academic_year_id UUID REFERENCES academic_years(id),
  created_at TIMESTAMPTZ DEFAULT NOW()
);

```

```

-- =====
-- COURS & DOCUMENTS

```

```
-- =====  
  
CREATE TYPE file_type AS ENUM ('pdf', 'video', 'image', 'document', 'audio', 'other');  
CREATE TYPE processing_status AS ENUM ('pending', 'processing', 'indexed', 'failed');
```

```
CREATE TABLE courses (  
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
    teacher_id UUID REFERENCES users(id) ON DELETE SET NULL,  
    subject_id UUID REFERENCES subjects(id),  
    class_id UUID REFERENCES classes(id),  
    academic_year_id UUID REFERENCES academic_years(id),  
    title VARCHAR(255) NOT NULL,  
    description TEXT,  
    chapter_number INTEGER,  
    tags TEXT[] DEFAULT '{}',  
    is_published BOOLEAN DEFAULT FALSE,  
    published_at TIMESTAMPTZ,  
    created_at TIMESTAMPTZ DEFAULT NOW(),  
    updated_at TIMESTAMPTZ DEFAULT NOW()  
);
```

```
CREATE TABLE course_files (  
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
    course_id UUID REFERENCES courses(id) ON DELETE CASCADE,  
    original_filename VARCHAR(255) NOT NULL,  
    stored_filename VARCHAR(255) NOT NULL,    -- nom dans MinIO  
    minio_bucket VARCHAR(100) NOT NULL,  
    minio_key TEXT NOT NULL,  
    file_type file_type NOT NULL,  
    file_size_bytes BIGINT,  
    mime_type VARCHAR(100),  
    ai_processing_status processing_status DEFAULT 'pending',  
    ai_chunk_count INTEGER DEFAULT 0,  
    ai_error_message TEXT,  
    uploaded_by UUID REFERENCES users(id),  
    created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

```
-- =====  
-- IA – EMBEDDINGS & CONVERSATIONS  
-- =====
```

```
CREATE TABLE document_chunks (  
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
    file_id UUID REFERENCES course_files(id) ON DELETE CASCADE,  
    course_id UUID REFERENCES courses(id) ON DELETE CASCADE,  
    subject_id UUID REFERENCES subjects(id),  
    class_id UUID REFERENCES classes(id),
```



```

    chunk_index INTEGER NOT NULL,
    content TEXT NOT NULL,
    token_count INTEGER,
    embedding vector(768),          -- nomic-embed-text dimension
    metadata JSONB DEFAULT '{}',
    created_at TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX idx_chunks_embedding ON document_chunks USING ivfflat (embedding
vector_cosine_ops)
    WITH (lists = 100);
CREATE INDEX idx_chunks_course ON document_chunks(course_id);
CREATE INDEX idx_chunks_class ON document_chunks(class_id);

CREATE TABLE ai_conversations (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    student_id UUID REFERENCES users(id) ON DELETE CASCADE,
    course_id UUID REFERENCES courses(id) ON DELETE SET NULL,
    subject_id UUID REFERENCES subjects(id),
    title VARCHAR(255),
    total_messages INTEGER DEFAULT 0,
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TYPE message_role AS ENUM ('user', 'assistant', 'system');

CREATE TABLE ai_messages (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    conversation_id UUID REFERENCES ai_conversations(id) ON DELETE CASCADE,
    role message_role NOT NULL,
    content TEXT NOT NULL,
    chunks_used UUID[],             -- IDs des chunks utilisés pour RAG
    confidence_score DECIMAL(3,2),  -- 0.00 à 1.00
    response_time_ms INTEGER,
    feedback SMALLINT,              -- 1 = positif, -1 = négatif, NULL = pas de feedback
    token_count INTEGER,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- =====
-- ÉVALUATIONS & NOTES
-- =====

CREATE TYPE evaluation_type AS ENUM ('quiz', 'devoir', 'examen', 'controle', 'oral', 'projet');

CREATE TABLE evaluations (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    title VARCHAR(255) NOT NULL,

```

```

description TEXT,
type evaluation_type NOT NULL,
subject_id UUID REFERENCES subjects(id),
class_id UUID REFERENCES classes(id),
teacher_id UUID REFERENCES users(id),
course_id UUID REFERENCES courses(id), -- optionnel : lié à un cours
max_score DECIMAL(5,2) DEFAULT 20.0,
coefficient DECIMAL(3,1) DEFAULT 1.0,
date DATE NOT NULL,
is_published BOOLEAN DEFAULT FALSE, -- notes visibles ou non
is_ai_generated BOOLEAN DEFAULT FALSE,
academic_year_id UUID REFERENCES academic_years(id),
created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE grades (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    evaluation_id UUID REFERENCES evaluations(id) ON DELETE CASCADE,
    student_id UUID REFERENCES users(id) ON DELETE CASCADE,
    score DECIMAL(5,2),
    comment TEXT,
    is_absent BOOLEAN DEFAULT FALSE,
    graded_by UUID REFERENCES users(id),
    graded_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ DEFAULT NOW(),
    UNIQUE(evaluation_id, student_id)
);

-- =====
-- QUIZZ ADAPTATIFS
-- =====

CREATE TYPE difficulty_level AS ENUM ('remediation', 'normal', 'advanced');

CREATE TABLE quizzes (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    course_id UUID REFERENCES courses(id) ON DELETE CASCADE,
    title VARCHAR(255) NOT NULL,
    difficulty difficulty_level DEFAULT 'normal',
    is_ai_generated BOOLEAN DEFAULT TRUE,
    time_limit_seconds INTEGER, -- NULL = pas de limite
    created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE quiz_questions (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    quiz_id UUID REFERENCES quizzes(id) ON DELETE CASCADE,
    question_text TEXT NOT NULL,

```

```

question_type VARCHAR(20) DEFAULT 'mcq', -- 'mcq', 'true_false', 'open'
options JSONB, -- [{text, is_correct}]
correct_answer TEXT,
explanation TEXT,
points INTEGER DEFAULT 1,
order_index INTEGER,
created_at TIMESTAMPTZ DEFAULT NOW()
);

```

```

CREATE TABLE quiz_attempts (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  quiz_id UUID REFERENCES quizzes(id) ON DELETE CASCADE,
  student_id UUID REFERENCES users(id) ON DELETE CASCADE,
  answers JSONB NOT NULL DEFAULT '[]', -- [{question_id, answer, is_correct}]
  score DECIMAL(5,2),
  max_score DECIMAL(5,2),
  duration_seconds INTEGER,
  completed_at TIMESTAMPTZ,
  adaptive_level_before difficulty_level,
  adaptive_level_after difficulty_level,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

```

```

-- =====
-- ABSENCES
-- =====

```

```

CREATE TYPE absence_type AS ENUM ('absence', 'retard', 'exclusion');
CREATE TYPE justification_status AS ENUM ('pending', 'justified', 'unjustified');

```

```

CREATE TABLE absences (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  student_id UUID REFERENCES users(id) ON DELETE CASCADE,
  class_id UUID REFERENCES classes(id),
  subject_id UUID REFERENCES subjects(id),
  teacher_id UUID REFERENCES users(id),
  type absence_type DEFAULT 'absence',
  date DATE NOT NULL,
  start_time TIME,
  end_time TIME,
  justification_status justification_status DEFAULT 'pending',
  justification_text TEXT,
  justification_document_url TEXT,
  notified_parent BOOLEAN DEFAULT FALSE,
  notified_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

```

```

-- =====
-- MESSAGERIE
-- =====

CREATE TABLE message_threads (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    subject VARCHAR(255),
    is_group BOOLEAN DEFAULT FALSE,
    created_by UUID REFERENCES users(id),
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE thread_participants (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    thread_id UUID REFERENCES message_threads(id) ON DELETE CASCADE,
    user_id UUID REFERENCES users(id) ON DELETE CASCADE,
    last_read_at TIMESTAMPTZ,
    is_archived BOOLEAN DEFAULT FALSE,
    UNIQUE(thread_id, user_id)
);

CREATE TABLE messages (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    thread_id UUID REFERENCES message_threads(id) ON DELETE CASCADE,
    sender_id UUID REFERENCES users(id) ON DELETE SET NULL,
    content TEXT NOT NULL,           -- chiffré AES-256 en base
    content_iv TEXT NOT NULL,       -- IV pour déchiffrement
    attachment_url TEXT,
    is_read BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- =====
-- NOTIFICATIONS
-- =====

CREATE TYPE notification_type AS ENUM (
    'absence', 'grade_published', 'message', 'ai_response',
    'quiz_available', 'event', 'system', 'decrochage_alert'
);

CREATE TABLE notifications (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID REFERENCES users(id) ON DELETE CASCADE,
    type notification_type NOT NULL,
    title VARCHAR(255) NOT NULL,
    body TEXT,

```

```

data JSONB DEFAULT '{}',          -- payload spécifique au type
is_read BOOLEAN DEFAULT FALSE,
read_at TIMESTAMPTZ,
push_sent BOOLEAN DEFAULT FALSE,
push_sent_at TIMESTAMPTZ,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- =====
-- GAMIFICATION
-- =====

CREATE TABLE student_profiles (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  student_id UUID REFERENCES users(id) ON DELETE CASCADE UNIQUE,
  xp_total INTEGER DEFAULT 0,
  level INTEGER DEFAULT 1,
  streak_days INTEGER DEFAULT 0,
  last_activity_date DATE,
  adaptive_level difficulty_level DEFAULT 'normal',
  adaptive_score DECIMAL(4,3) DEFAULT 0.500,    -- 0.000 à 1.000
  avatar_config JSONB DEFAULT '{}',
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE badges (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  code VARCHAR(50) UNIQUE NOT NULL,
  name VARCHAR(100) NOT NULL,
  description TEXT,
  icon_url TEXT,
  xp_reward INTEGER DEFAULT 0,
  condition_type VARCHAR(50),          -- 'streak', 'quiz_score', 'login', etc.
  condition_value INTEGER,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE student_badges (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  student_id UUID REFERENCES users(id) ON DELETE CASCADE,
  badge_id UUID REFERENCES badges(id),
  awarded_at TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE(student_id, badge_id)
);

CREATE TABLE xp_transactions (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),

```

```

student_id UUID REFERENCES users(id) ON DELETE CASCADE,
amount INTEGER NOT NULL,           -- positif ou négatif
reason VARCHAR(100) NOT NULL,
reference_id UUID,                 -- ID de quiz, cours, etc.
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- =====
-- ÉVÉNEMENTS SCOLAIRES
-- =====

CREATE TABLE school_events (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    title VARCHAR(255) NOT NULL,
    description TEXT,
    event_type VARCHAR(50),          -- 'exam', 'holiday', 'meeting', 'activity'
    start_datetime TIMESTAMPTZ NOT NULL,
    end_datetime TIMESTAMPTZ,
    all_day BOOLEAN DEFAULT FALSE,
    target_classes UUID[],           -- NULL = toute l'école
    created_by UUID REFERENCES users(id),
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- =====
-- INDEXES SUPPLÉMENTAIRES
-- =====

CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_users_role ON users(role);
CREATE INDEX idx_grades_student ON grades(student_id);
CREATE INDEX idx_grades_evaluation ON grades(evaluation_id);
CREATE INDEX idx_absences_student ON absences(student_id);
CREATE INDEX idx_absences_date ON absences(date);
CREATE INDEX idx_notifications_user ON notifications(user_id, is_read);
CREATE INDEX idx_messages_thread ON messages(thread_id, created_at);
CREATE INDEX idx_quiz_attempts_student ON quiz_attempts(student_id);
CREATE INDEX idx_ai_messages_conversation ON ai_messages(conversation_id, created_at);

```

4. Profils Utilisateurs & Permissions

4.1 Matrice de permissions complète

Permission	Admin	Enseignant	Élève	Parent
Créer/supprimer utilisateur	✓	✗	✗	✗
Gérer classes & inscriptions	✓	✗	✗	✗
Voir tous les utilisateurs	✓	✗	✗	✗
Créer emploi du temps	✓	✗	✗	✗
Voir emploi du temps (sa classe)	✓	✓	✓	✓ (enfant)
Déposer cours/fichiers	✓	✓ (sa matière)	✗	✗
Voir cours	✓	✓	✓ (sa classe)	✗
Créer évaluation	✓	✓ (sa matière)	✗	✗
Saisir notes	✓	✓ (sa classe)	✗	✗
Publier notes	✓	✓ (ses évals)	✗	✗
Voir ses notes	✓	✓ (sa classe)	✓	✓ (enfant)
Voir notes d'autres élèves	✓	✓ (sa classe)	✗	✗
Saisir absences	✓	✓ (sa classe)	✗	✗
Justifier absences	✓	✓	✗	✓ (enfant)
Voir absences	✓	✓ (sa classe)	✓ (les siennes)	✓ (enfant)
Accès Mentor IA	✓	✓	✓	✗

Permission	Admin	Enseignant	Élève	Parent
Générer quiz IA	✓	✓	✗	✗
Passer quiz	✗	✗	✓	✗
Envoyer message	✓	✓	⚠ (vers enseignant uniquement)	✓ (vers enseignant/admin)
Tableau de bord décrochage	✓	✓ (sa classe)	✗	✗
Voir rapports statistiques	✓	✓ (ses classes)	✗	✗
Exporter données (CSV/PDF)	✓	✓ (ses classes)	✗	✗
Gérer événements scolaires	✓	✗	✗	✗
Voir logs d'audit	✓	✗	✗	✗

5. Stack Technique Complète

5.1 Versions exactes à utiliser

Composant	Technologie	Version	Rôle
Backend API	FastAPI	0.115.x	API REST + WebSocket
ORM	SQLAlchemy	2.0.x	Async ORM
Migrations	Alembic	1.13.x	Gestion schéma DB
Validation	Pydantic	2.x	Schémas & validation
Base de données	PostgreSQL	16	DB principale
Vecteurs IA	pgvector	0.7.x	Embeddings RAG
Cache / Sessions	Redis	7.2	Cache + pub/sub
Queue async	Celery	5.3.x	Tâches background
Monitoring queue	Flower	2.0.x	UI monitoring Celery
Auth	python-jose	3.3.x	JWT
Hachage	bcrypt	4.x	Passwords
TOTP (MFA)	pyotp	2.9.x	MFA admin/enseignant
PDF extraction	PyMuPDF	1.24.x	Extraction texte cours
HTTP client	httpx	0.27.x	Appels Ollama
LLM local	Ollama	latest	Inférence IA
Modèle LLM	qwen2.5:7b-instruct	latest	Chat + génération
Modèle embed	nomic-embed-text	latest	Embeddings RAG
Stockage fichiers	MinIO	latest	S3-compatible
Frontend Web	React	18.x	UI web
Build tool	Vite	5.x	Bundler rapide
State management	TanStack Query	5.x	State serveur
Forms	React Hook Form	7.x	Formulaires
UI Components	shadcn/ui + Tailwind	latest	Design system
i18n	react-i18next	15.x	FR/AR
Mobile	React Native + Expo	SDK 52	iOS + Android

Composant	Technologie	Version	Rôle
Notifications push	Expo Notifications	latest	Push mobile
Reverse proxy	Nginx	1.25	TLS + proxy
Monitoring	Prometheus + Grafana	latest	Métriques
Logs	Loki	latest	Agrégation logs
Conteneurs	Docker + Compose	27.x	Déploiement

6. Structure du Projet

```

don-bosco-connect/
├── README.md
├── docker-compose.yml
├── docker-compose.dev.yml          # override dev (hot reload)
├── .env.example
├── .gitignore
├──
├── backend/
│   ├── Dockerfile
│   ├── requirements.txt
│   ├── pyproject.toml
│   ├── alembic.ini
│   ├── alembic/
│   │   └── versions/              # fichiers de migration
│   └── app/
│       ├── main.py               # entrypoint FastAPI
│       ├── config.py             # settings (pydantic-settings)
│       ├── database.py           # connexion async PostgreSQL
│       ├── redis_client.py
│       └── minio_client.py
│
│   ├── models/                  # SQLAlchemy ORM models
│   │   ├── user.py
│   │   ├── academic.py
│   │   ├── course.py
│   │   ├── evaluation.py
│   │   ├── ai.py
│   │   ├── gamification.py
│   │   └── messaging.py
│   └──
│       ├── schemas/             # Pydantic schemas (request/response)
│       │   ├── auth.py
│       │   ├── user.py
│       │   ├── course.py
│       │   ├── evaluation.py
│       │   ├── ai.py
│       │   └── gamification.py
│       └── api/
│           ├── deps.py           # Dependencies (get_current_user, etc.)
│           └── v1/
│               ├── router.py     # agrégateur des routers
│               ├── auth.py
│               ├── users.py
│               ├── classes.py
│               ├── courses.py
│               ├── evaluations.py
│               ├── absences.py
│               ├── messages.py
│               └── notifications.py

```

```
├── ai.py # chat + quiz generation
├── gamification.py
└── websocket.py

├── services/ # logique métier
│   ├── auth_service.py
│   ├── grade_service.py
│   ├── absence_service.py
│   ├── rag_service.py # RAG pipeline
│   ├── adaptive_service.py # algorithme adaptatif
│   ├── gamification_service.py
│   └── notification_service.py
├── workers/ # tâches Celery
│   ├── celery_app.py
│   ├── document_tasks.py # ingestion PDF → embeddings
│   ├── notification_tasks.py
│   └── ai_tasks.py # génération quiz background
├── core/
│   ├── security.py # JWT, bcrypt, chiffrement
│   ├── permissions.py # décorateurs RBAC
│   └── exceptions.py
├── tests/
│   ├── conftest.py
│   ├── test_auth.py
│   ├── test_courses.py
│   ├── test_rag.py
│   └── test_adaptive.py
├── frontend/
│   ├── Dockerfile
│   ├── package.json
│   ├── vite.config.ts
│   ├── tailwind.config.ts
│   ├── public/
│   └── src/
│       ├── main.tsx
│       ├── App.tsx
│       ├── i18n/
│       │   ├── fr.json
│       │   └── ar.json
│       ├── lib/
│       │   ├── api.ts # axios instance + interceptors
│       │   ├── auth.ts # JWT storage (memory only)
│       │   └── ws.ts # WebSocket manager
│       ├── store/
│       │   └── authStore.ts # Zustand auth store
│       ├── components/
│       │   └── ui/ # shadcn/ui components
```

```
|
|
|   └─ layout/
|       └─ Sidebar.tsx
|       └─ Header.tsx
|       └─ Layout.tsx
|
|   └─ AIChat/ # Mentor IA composant
|   └─ GradeBook/
|   └─ QuizPlayer/
|   └─ Gamification/
|
|   └─ pages/
|       └─ auth/ # login, mfa
|       └─ admin/
|       └─ teacher/
|       └─ student/
|       └─ parent/
|
|   └─ types/ # TypeScript interfaces
|
└─ mobile/
    └─ app.json
    └─ package.json
    └─ src/
        └─ app/ # Expo Router (file-based)
            └─ (auth)/
                └─ login.tsx
                └─ mfa.tsx
            └─ (admin)/
            └─ (teacher)/
            └─ (student)/
            └─ (parent)/
        └─ components/
        └─ lib/
        └─ hooks/
|
└─ nginx/
    └─ Dockerfile
    └─ nginx.conf
|
└─ monitoring/
    └─ prometheus.yml
    └─ grafana/
        └─ dashboards/
    └─ loki-config.yml
|
└─ scripts/
    └─ init_db.py # seed données initiales
    └─ create_admin.py
    └─ backup.sh
```

7. Configuration Docker Compose

```

# docker-compose.yml
version: '3.9'

networks:
  donbosco_net:
    driver: bridge

volumes:
  postgres_data:
  redis_data:
  minio_data:
  ollama_data:
  grafana_data:
  prometheus_data:
  loki_data:

services:

# — BASE DE DONNÉES —————
db:
  image: pgvector/pgvector:pg16
  container_name: donbosco_db
  restart: unless-stopped
  environment:
    POSTGRES_DB: ${DB_NAME}
    POSTGRES_USER: ${DB_USER}
    POSTGRES_PASSWORD: ${DB_PASSWORD}
  volumes:
    - postgres_data:/var/lib/postgresql/data
    - ./scripts/init_db.sql:/docker-entrypoint-initdb.d/init.sql
  networks: [donbosco_net]
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U ${DB_USER} -d ${DB_NAME}"]
    interval: 10s
    timeout: 5s
    retries: 5

# — REDIS —————
redis:
  image: redis:7.2-alpine
  container_name: donbosco_redis
  restart: unless-stopped
  command: redis-server --requirepass ${REDIS_PASSWORD} --maxmemory 512mb --maxmemory-policy
allkeys-lru
  volumes:
    - redis_data:/data
  networks: [donbosco_net]

```

```
healthcheck:
  test: ["CMD", "redis-cli", "--no-auth-warning", "-a", "${REDIS_PASSWORD}", "ping"]
  interval: 10s
  timeout: 5s
  retries: 5
```

— MINIO —————

```
minio:
  image: minio/minio:latest
  container_name: donbosco_minio
  restart: unless-stopped
  command: server /data --console-address ":9001"
  environment:
    MINIO_ROOT_USER: ${MINIO_ROOT_USER}
    MINIO_ROOT_PASSWORD: ${MINIO_ROOT_PASSWORD}
  volumes:
    - minio_data:/data
  ports:
    - "9001:9001" # console admin (interne réseau école seulement)
  networks: [donbosco_net]
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost:9000/minio/health/live"]
    interval: 30s
    timeout: 20s
    retries: 3
```

— OLLAMA —————

```
ollama:
  image: ollama/ollama:latest
  container_name: donbosco_ollama
  restart: unless-stopped
  volumes:
    - ollama_data:/root/.ollama
  networks: [donbosco_net]
  # Décommenter si GPU disponible :
  # deploy:
  #   resources:
  #     reservations:
  #       devices:
  #         - driver: nvidia
  #           count: 1
  #           capabilities: [gpu]
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost:11434/api/tags"]
    interval: 30s
    timeout: 10s
    retries: 5
```



```
# — API FASTAPI —————
api:
  build:
    context: ./backend
    dockerfile: Dockerfile
  container_name: donbosco_api
  restart: unless-stopped
  environment:
    - DATABASE_URL=postgresql+asyncpg://${DB_USER}:${DB_PASSWORD}@db:5432/${DB_NAME}
    - REDIS_URL=redis://:${REDIS_PASSWORD}@redis:6379/0
    - MINIO_ENDPOINT=minio:9000
    - MINIO_ACCESS_KEY=${MINIO_ROOT_USER}
    - MINIO_SECRET_KEY=${MINIO_ROOT_PASSWORD}
    - OLLAMA_BASE_URL=http://ollama:11434
    - SECRET_KEY=${SECRET_KEY}
    - ENVIRONMENT=production
  depends_on:
    db: { condition: service_healthy }
    redis: { condition: service_healthy }
    minio: { condition: service_healthy }
  networks: [donbosco_net]
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost:8000/health"]
    interval: 30s
    timeout: 10s
    retries: 3
```

```
# — CELERY WORKER —————
worker:
  build:
    context: ./backend
    dockerfile: Dockerfile
  container_name: donbosco_worker
  restart: unless-stopped
  command: celery -A app.workers.celery_app worker --loglevel=info --concurrency=4
  environment:
    - DATABASE_URL=postgresql+asyncpg://${DB_USER}:${DB_PASSWORD}@db:5432/${DB_NAME}
    - REDIS_URL=redis://:${REDIS_PASSWORD}@redis:6379/0
    - MINIO_ENDPOINT=minio:9000
    - MINIO_ACCESS_KEY=${MINIO_ROOT_USER}
    - MINIO_SECRET_KEY=${MINIO_ROOT_PASSWORD}
    - OLLAMA_BASE_URL=http://ollama:11434
  depends_on:
    - db
    - redis
    - ollama
  networks: [donbosco_net]
```

```
# — CELERY BEAT (scheduler) —————
beat:
  build:
    context: ./backend
    dockerfile: Dockerfile
  container_name: donbosco_beat
  restart: unless-stopped
  command: celery -A app.workers.celery_app beat --loglevel=info
  environment:
    - REDIS_URL=redis://:${REDIS_PASSWORD}@redis:6379/0
  depends_on:
    - redis
  networks: [donbosco_net]
```

```
# — FLOWER (monitoring Celery) —————
flower:
  build:
    context: ./backend
    dockerfile: Dockerfile
  container_name: donbosco_flower
  restart: unless-stopped
  command: celery -A app.workers.celery_app flower --port=5555
  ports:
    - "5555:5555" # accès réseau interne uniquement
  depends_on:
    - redis
    - worker
  networks: [donbosco_net]
```

```
# — FRONTEND WEB —————
frontend:
  build:
    context: ./frontend
    dockerfile: Dockerfile
  container_name: donbosco_frontend
  restart: unless-stopped
  networks: [donbosco_net]
```

```
# — NGINX —————
nginx:
  build:
    context: ./nginx
    dockerfile: Dockerfile
  container_name: donbosco_nginx
  restart: unless-stopped
  ports:
    - "80:80"
    - "443:443"
```

```
depends_on:
  - api
  - frontend
volumes:
  - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
  - ./nginx/ssl:/etc/nginx/ssl:ro
networks: [donbosco_net]

# — PROMETHEUS —————
prometheus:
  image: prom/prometheus:latest
  container_name: donbosco_prometheus
  restart: unless-stopped
  volumes:
    - ./monitoring/prometheus.yml:/etc/prometheus/prometheus.yml:ro
    - prometheus_data:/prometheus
  networks: [donbosco_net]

# — GRAFANA —————
grafana:
  image: grafana/grafana:latest
  container_name: donbosco_grafana
  restart: unless-stopped
  environment:
    GF_SECURITY_ADMIN_PASSWORD: ${GRAFANA_PASSWORD}
    GF_USERS_ALLOW_SIGN_UP: "false"
  volumes:
    - grafana_data:/var/lib/grafana
    - ./monitoring/grafana/dashboards:/etc/grafana/provisioning/dashboards
  ports:
    - "3001:3000" # accès réseau interne uniquement
  networks: [donbosco_net]
```

8. Variables d'Environnement

.env.example – copier en .env et remplir toutes les valeurs

— APPLICATION —————

ENVIRONMENT=production # 'development' | 'production'
SECRET_KEY= # 64 chars random hex: openssl rand -hex 32
ACCESS_TOKEN_EXPIRE_MINUTES=15
REFRESH_TOKEN_EXPIRE_DAYS=7
ALGORITHM=HS256

— BASE DE DONNÉES —————

DB_NAME=donbosco
DB_USER=donbosco_user
DB_PASSWORD= # mot de passe fort
DB_HOST=db
DB_PORT=5432

— REDIS —————

REDIS_PASSWORD= # mot de passe fort
REDIS_HOST=redis
REDIS_PORT=6379

— MINIO —————

MINIO_ROOT_USER=donbosco_admin
MINIO_ROOT_PASSWORD= # mot de passe fort
MINIO_ENDPOINT=minio:9000
MINIO_BUCKET_COURSES=courses
MINIO_BUCKET_AVATARS=avatars
MINIO_SECURE=false # true si HTTPS

— OLLAMA —————

OLLAMA_BASE_URL=http://ollama:11434
OLLAMA_CHAT_MODEL=qwen2.5:7b-instruct
OLLAMA_EMBED_MODEL=nomic-embed-text
OLLAMA_MAX_CONCURRENT_SESSIONS=10
OLLAMA_MAX_TOKENS_PER_SESSION=2000

— IA PARAMÈTRES —————

RAG_TOP_K=5 # nombre de chunks retournés
RAG_SIMILARITY_THRESHOLD=0.70 # seuil cosine similarity
CHUNK_SIZE=512 # taille chunk en tokens
CHUNK_OVERLAP=64 # overlap entre chunks
AI_DAILY_TOKEN_LIMIT_PER_STUDENT=10000

— MONITORING —————

GRAFANA_PASSWORD= # mot de passe fort

— SÉCURITÉ —————

```
MAX_LOGIN_ATTEMPTS=5
LOGIN_LOCKOUT_MINUTES=15
CORS_ORIGINS=https://donbosco.local
ENCRYPTION_KEY= # 32 bytes pour AES-256 messages

# — UPLOAD —
MAX_UPLOAD_SIZE_MB=200
ALLOWED_MIME_TYPES=application/pdf,video/mp4,video/webm,image/jpeg,image/png,application/msword
,application/vnd.openxmlformats-officedocument.wordprocessingml.document
```

9. Spécification API REST

9.1 Conventions générales

- Base URL : `/api/v1/`
- Format : JSON
- Auth : `Authorization: Bearer <access_token>` (sauf endpoints publics)
- Pagination : `?page=1&per_page=20` → `{items, total, page, per_page, pages}`
- Erreurs : `{error: {code, message, details}}`
- Versioning : URL path (`/v1/` , `/v2/`)

9.2 Endpoints complets

— AUTH —

POST	/api/v1/auth/login	Corps: {email, password}
POST	/api/v1/auth/refresh	Corps: {refresh_token}
POST	/api/v1/auth/logout	Header: Authorization
POST	/api/v1/auth/mfa/setup	Génère QR code TOTP
POST	/api/v1/auth/mfa/verify	Corps: {code}
POST	/api/v1/auth/password/change	Corps: {old_password, new_password}

— UTILISATEURS —

GET	/api/v1/users/me	Profil courant
PATCH	/api/v1/users/me	Mise à jour profil
GET	/api/v1/users	[Admin] Liste paginée ?role=&search=
POST	/api/v1/users	[Admin] Créer utilisateur
GET	/api/v1/users/{id}	[Admin] Détails utilisateur
PATCH	/api/v1/users/{id}	[Admin] Modifier
DELETE	/api/v1/users/{id}	[Admin] Désactiver (soft delete)
POST	/api/v1/users/bulk-import	[Admin] Import CSV élèves

— STRUCTURE SCOLAIRE —

GET	/api/v1/academic-years	Liste années scolaires
POST	/api/v1/academic-years	[Admin] Créer
GET	/api/v1/classes	?academic_year_id=&teacher_id=
POST	/api/v1/classes	[Admin]
GET	/api/v1/classes/{id}	
GET	/api/v1/classes/{id}/students	Liste élèves de la classe
POST	/api/v1/classes/{id}/enroll	[Admin] Inscrire élève
DELETE	/api/v1/classes/{id}/students/{student_id}	[Admin]
GET	/api/v1/subjects	Liste matières
POST	/api/v1/subjects	[Admin]
GET	/api/v1/timetable	?class_id=&academic_year_id=
POST	/api/v1/timetable/slots	[Admin] Créer créneau

— COURS & FICHIERS —

GET	/api/v1/courses	?class_id=&subject_id=&teacher_id=
POST	/api/v1/courses	[Teacher/Admin]
GET	/api/v1/courses/{id}	
PATCH	/api/v1/courses/{id}	[Teacher owner/Admin]
DELETE	/api/v1/courses/{id}	[Teacher owner/Admin]
POST	/api/v1/courses/{id}/publish	Rendre visible aux élèves
POST	/api/v1/courses/{id}/files	Upload fichier (multipart/form-data)
GET	/api/v1/courses/{id}/files	Liste fichiers
DELETE	/api/v1/courses/{id}/files/{file_id}	
GET	/api/v1/courses/{id}/files/{file_id}/download	URL présignée MinIO

— ÉVALUATIONS & NOTES —

GET	/api/v1/evaluations	?class_id=&subject_id=&type=
POST	/api/v1/evaluations	[Teacher/Admin]
GET	/api/v1/evaluations/{id}	
PATCH	/api/v1/evaluations/{id}	
DELETE	/api/v1/evaluations/{id}	

GET /api/v1/evaluations/{id}/grades
POST /api/v1/evaluations/{id}/grades [Teacher] Saisie notes bulk
PATCH /api/v1/evaluations/{id}/grades/{student_id}
POST /api/v1/evaluations/{id}/publish
GET /api/v1/students/{id}/grades ?subject_id=&academic_year_id=
GET /api/v1/classes/{id}/report Bulletin complet classe (PDF stream)
GET /api/v1/students/{id}/report Bulletin élève (PDF stream)

— ABSENCES —

GET /api/v1/absences ?student_id=&class_id=&date=
POST /api/v1/absences [Teacher/Admin] Saisir absence
GET /api/v1/absences/{id}
PATCH /api/v1/absences/{id}/justify [Parent/Admin]
GET /api/v1/students/{id}/absences ?from=&to=

— MESSAGERIE —

GET /api/v1/messages/threads Mes fils de discussion
POST /api/v1/messages/threads Créer fil
GET /api/v1/messages/threads/{id} Messages du fil (paginé)
POST /api/v1/messages/threads/{id} Envoyer message
PATCH /api/v1/messages/threads/{id}/read Marquer lu

— NOTIFICATIONS —

GET /api/v1/notifications ?is_read=false&type=
PATCH /api/v1/notifications/{id}/read
PATCH /api/v1/notifications/read-all
DELETE /api/v1/notifications/{id}

— IA – MENTOR & QUIZ —

GET /api/v1/ai/conversations Mes conversations (étudiant)
POST /api/v1/ai/conversations Créer nouvelle conversation
GET /api/v1/ai/conversations/{id} Historique messages
POST /api/v1/ai/conversations/{id}/messages Envoyer message → SSE stream
POST /api/v1/ai/conversations/{id}/messages/{msg_id}/feedback 👍/👎
POST /api/v1/ai/quiz/generate [Teacher] Générer quiz depuis cours
GET /api/v1/quizzes ?course_id=&difficulty=
GET /api/v1/quizzes/{id}
POST /api/v1/quizzes/{id}/attempt [Student] Soumettre réponses
GET /api/v1/quizzes/{id}/attempts [Student] Mes tentatives

— GAMIFICATION —

GET /api/v1/gamification/profile [Student] Mon profil XP/badges
GET /api/v1/gamification/badges Tous les badges disponibles
GET /api/v1/gamification/leaderboard ?class_id=&period=week|month
GET /api/v1/gamification/xp-history [Student] Historique XP

— ANALYTICS / ADMIN —

GET /api/v1/analytics/dashboard [Admin] Métriques globales
GET /api/v1/analytics/at-risk [Admin/Teacher] Élèves à risque décrochage
GET /api/v1/analytics/class/{id} [Admin/Teacher] Stats détaillées classe

GET	/api/v1/analytics/subjects	[Admin] Performance par matière
— ÉVÉNEMENTS —		
GET	/api/v1/events	?from=&to=&class_id=
POST	/api/v1/events	[Admin]
PATCH	/api/v1/events/{id}	[Admin]
DELETE	/api/v1/events/{id}	[Admin]
— SANTÉ —		
GET	/health	{status, db, redis, ollama, minio}
GET	/metrics	Prometheus metrics (réseau interne)

9.3 Exemples de schémas Pydantic

```
# schemas/ai.py
from pydantic import BaseModel
from typing import Optional

class ChatMessageRequest(BaseModel):
    content: str
    course_id: Optional[str] = None # filtrer RAG à un cours spécifique

class ChatMessageResponse(BaseModel):
    message_id: str
    conversation_id: str
    # Le contenu est streamé via SSE, pas dans ce schéma

class QuizGenerateRequest(BaseModel):
    course_id: str
    num_questions: int = 10 # 5 à 20
    difficulty: str = "normal" # remediation | normal | advanced
    question_types: list[str] = ["mcq", "true_false"]

class QuizSubmitRequest(BaseModel):
    answers: list[dict] # [{question_id, answer}]
    duration_seconds: int
```

10. Spécification WebSocket

10.1 Endpoint principal

```
wss://donbosco.local/ws/v1/{channel}?token={access_token}
```


10.2 Canaux disponibles

Canal	Accès	Description
/ws/v1/notifications	Tous	Notifications temps réel
/ws/v1/messages/{thread_id}	Participants	Chat en temps réel
/ws/v1/ai/stream/{conversation_id}	Élève/Enseignant	Streaming réponse IA
/ws/v1/admin/live	Admin	Métriques live

10.3 Format des messages

```
// Serveur → Client : notification
{
  "type": "notification",
  "data": {
    "id": "uuid",
    "notification_type": "absence",
    "title": "Absence signalée",
    "body": "Votre enfant Adam a été absent en Mathématiques ce matin.",
    "created_at": "2025-09-15T09:05:00Z"
  }
}

// Serveur → Client : token IA streaming
{
  "type": "ai_token",
  "data": {
    "token": "La",
    "message_id": "uuid",
    "is_final": false
  }
}

// Serveur → Client : fin streaming IA
{
  "type": "ai_done",
  "data": {
    "message_id": "uuid",
    "confidence_score": 0.87,
    "chunks_used": ["uuid1", "uuid2"],
    "is_final": true
  }
}

// Serveur → Client : nouveau message chat
{
  "type": "new_message",
  "data": {
    "thread_id": "uuid",
    "message_id": "uuid",
    "sender": { "id": "uuid", "name": "Prof. Hamdi", "role": "teacher" },
    "content": "...", // déchiffré côté client
    "created_at": "..."
  }
}
```

11. Moteur IA — Spécification Détaillée

11.1 Modèles Ollama à installer au démarrage

```
# Script d'initialisation Ollama (exécuter une fois)
ollama pull qwen2.5:7b-instruct      # Mentor IA + génération quiz
ollama pull nomic-embed-text        # Embeddings RAG
```

11.2 Algorithme d'apprentissage adaptatif

```
# services/adaptive_service.py

def compute_adaptive_score(
    quiz_score: float,      # 0.0 à 1.0 (score normalisé)
    response_time: float,   # secondes
    max_time: float,        # temps max du quiz en secondes
    historical_score: float # score lissé précédent
) -> float:
    """Calcule le score adaptatif lissé d'un élève."""

    # Normalisation vitesse : plus rapide = meilleur score
    speed_score = max(0.0, 1.0 - (response_time / max_time))

    # Score de la session courante
    session_score = (quiz_score * 0.70) + (speed_score * 0.30)

    # Lissage exponentiel (évite les variations brutales)
    # alpha = 0.40 → sessions récentes pèsent plus
    smoothed = (session_score * 0.40) + (historical_score * 0.60)

    return round(min(max(smoothed, 0.0), 1.0), 3)

def get_adaptive_level(score: float) -> str:
    if score < 0.40:
        return "remediation" # exercices de rattrapage
    elif score < 0.75:
        return "normal"
    else:
        return "advanced"    # exercices d'approfondissement
```

11.3 Pipeline RAG complet

```
# services/rag_service.py
```

```
SYSTEM_PROMPT = """Tu es le tuteur IA de {subject_name} au collège Don Bosco Tunis.  
Tu aides les élèves à comprendre leur programme de cours.
```

```
RÈGLES STRICTES :
```

1. Réponds UNIQUEMENT en te basant sur les documents fournis dans le contexte.
 2. Si la réponse n'est pas dans les documents, dis : "Je ne trouve pas cette information dans tes cours. Demande à ton professeur."
 3. Réponds en français, sauf si l'élève écrit en arabe.
 4. Adapte ton niveau de langage pour un collégien de {class_level}.
 5. Sois encourageant et pédagogique.
 6. Ne génère JAMAIS de contenu hors programme scolaire.
- ```
"""
```

```
async def query_rag(
 question: str,
 student_id: str,
 course_id: Optional[str],
 conversation_history: list[dict]
) -> AsyncGenerator[str, None]:
 """Stream une réponse RAG."""

 # 1. Embed la question
 question_embedding = await embed_text(question)

 # 2. Recherche de similarité pgvector
 chunks = await search_similar_chunks(
 embedding=question_embedding,
 course_id=course_id,
 top_k=RAG_TOP_K,
 similarity_threshold=RAG_SIMILARITY_THRESHOLD
)

 if not chunks:
 yield "Je ne trouve pas cette information dans tes cours. Demande à ton professeur."
 return

 # 3. Construction du contexte
 context = "\n\n--\n\n".join([
 f"[Source: {chunk.metadata.get('filename', 'cours')}] \n{chunk.content}"
 for chunk in chunks
])

 # 4. Construction du prompt
 messages = [
 {"role": "system", "content": SYSTEM_PROMPT.format(...)},
```

```

Historique de conversation (max 10 derniers messages)
*conversation_history[-10:],
{
 "role": "user",
 "content": f"CONTEXTE DES COURS:\n{context}\n\nQUESTION: {question}"
}
]

5. Stream de réponse Ollama
async for token in stream_ollama(
 model=OLLAMA_CHAT_MODEL,
 messages=messages,
 max_tokens=1000,
 temperature=0.3 # bas pour rester factuel
):
 yield token

```

## 11.4 Génération de quiz par IA

```
QUIZ_GENERATION_PROMPT = """Génère {num_questions} questions de quiz sur ce cours.
```

```
COURS :
{course_content}
```

```
FORMAT DE RÉPONSE (JSON uniquement, aucun texte autour) :
```

```
{{
 "questions": [
 {{
 "question": "...",
 "type": "mcq",
 "options": [
 {{ "text": "...", "is_correct": true}},
 {{ "text": "...", "is_correct": false}},
 {{ "text": "...", "is_correct": false}},
 {{ "text": "...", "is_correct": false}}
],
 "explanation": "Explication de la bonne réponse",
 "difficulty": "{difficulty}"
 }}
]
}}
```

```
CONSIGNES :
```

- Difficulté : {difficulty} ({difficulty\_description})
  - Types demandés : {question\_types}
  - Questions claires, sans ambiguïté
  - Une seule bonne réponse par QCM
  - Explications pédagogiques
- ```
"""
```

11.5 Détection de décrochage

```
# services/adaptive_service.py

def compute_dropout_risk(student_data: dict) -> float:
    """
    Retourne un score de risque entre 0.0 (faible) et 1.0 (élevé).
    Déclenche une alerte admin si score > 0.65.
    """
    score = 0.0

    # Absences (poids 35%)
    absence_rate = student_data["absences_last_30d"] / 20 # 20 jours ouvrables
    score += min(absence_rate, 1.0) * 0.35

    # Niveau adaptatif (poids 30%)
    if student_data["adaptive_level"] == "remediation":
        score += 0.30
    elif student_data["adaptive_level"] == "normal":
        score += 0.10

    # Engagement IA (poids 20%)
    ai_sessions = student_data["ai_sessions_last_14d"]
    if ai_sessions == 0:
        score += 0.20
    elif ai_sessions < 2:
        score += 0.10

    # Streak (poids 15%)
    if student_data["streak_days"] == 0:
        score += 0.15
    elif student_data["streak_days"] < 3:
        score += 0.07

    return round(min(score, 1.0), 2)
```

11.6 Système de gamification

```

XP_REWARDS = {
    "daily_login": 5,
    "quiz_completed": 10,
    "quiz_perfect_score": 50,
    "quiz_above_80": 20,
    "ai_session": 2,
    "streak_3_days": 25,
    "streak_7_days": 75,
    "streak_30_days": 300,
    "course_viewed": 1,
    "first_quiz": 15,      # badge déclencheur
}

LEVEL_THRESHOLDS = [
    (0, "Débutant"), (100, "Apprenti"), (300, "Étudiant"),
    (600, "Assidu"), (1000, "Expert"), (1500, "Maître"),
    (2500, "Légende")
]

BADGES = [
    {"code": "first_login",      "name": "Première Connexion",      "condition_type": "login",
    "condition_value": 1},
    {"code": "quiz_master",      "name": "Maître des Quiz",          "condition_type":
    "quiz_score", "condition_value": 100},
    {"code": "streak_7",         "name": "7 Jours d'Affilée",        "condition_type": "streak",
    "condition_value": 7},
    {"code": "streak_30",        "name": "Un Mois Sans Pause",       "condition_type": "streak",
    "condition_value": 30},
    {"code": "ai_explorer",      "name": "Explorateur IA",          "condition_type":
    "ai_sessions", "condition_value": 10},
    {"code": "perfectionist",    "name": "Perfectionniste",         "condition_type":
    "perfect_quizzes", "condition_value": 5},
    {"code": "knowledge_seeker", "name": "Chercheur de Savoir",      "condition_type":
    "courses_viewed", "condition_value": 20},
]

```

12. User Stories & Critères d'Acceptation

12.1 Priorité P0 — Critique pour MVP

US-01 — Connexion sécurisée

En tant qu'utilisateur, je veux me connecter avec email/mot de passe + MFA (admin/enseignant) pour accéder à ma section.

Critères :

- ☐ Connexion retourne access_token (15min) + refresh_token (7j)
 - ☐ 5 échecs = blocage 15 minutes
 - ☐ Admin et enseignants ont MFA TOTP obligatoire
 - ☐ Token invalide → redirection vers login
 - ☐ Déconnexion révoque le refresh token dans Redis
-

US-02 — Notification absence temps réel (Parent)

En tant que parent, je veux recevoir une notification sur mon téléphone dans les 5 minutes si mon enfant est absent.

Critères :

- ☐ Enseignant saisit absence → notification WebSocket déclenchée < 5 min
 - ☐ Notification push Expo si app mobile installée
 - ☐ Notification indique : matière, heure, enseignant
 - ☐ Parent peut justifier via l'app
 - ☐ Historique des absences accessible depuis l'app parent
-

US-03 — Dépôt et indexation de cours (Enseignant)

En tant qu'enseignant, je veux uploader un PDF de cours et qu'il soit automatiquement indexé pour le Mentor IA.

Critères :

- ☐ Upload jusqu'à 200 Mo (PDF, DOCX, MP4)
 - ☐ Barre de progression pendant upload
 - ☐ Statut en temps réel : "En traitement...", "Indexé ✓", "Erreur ✗"
 - ☐ Indexation complète < 60 secondes pour PDF 50 Mo
 - ☐ Confirmation : nombre de chunks créés affiché
 - ☐ En cas d'erreur : message explicite + option de réessai
-

US-04 — Mentor IA 24/7 (Élève)

En tant qu'élève, je veux poser une question sur un cours à n'importe quelle heure et obtenir une réponse basée sur le cours de mon prof.

Critères :

- ☐ Réponse commence à streamer < 3 secondes
 - ☐ Réponse cite la source (fichier du cours)
 - ☐ Si pas de réponse dans les documents → message explicite d'orientation
 - ☐ Historique des conversations conservé
 - ☐ Bouton 👍/👎 sur chaque réponse
 - ☐ Limite : 10 000 tokens/élève/jour (compteur visible)
-

US-05 — Saisie et consultation des notes

En tant qu'enseignant, je veux saisir les notes d'une évaluation et les publier quand je suis prêt.

Critères :

- ☐ Saisie bulk : une note par ligne, autocomplétion nom élève
 - ☐ Calcul automatique : moyenne, min, max, distribution
 - ☐ Notes non visibles aux élèves/parents tant que non publiées
 - ☐ Publication instantanée → notification aux élèves et parents
 - ☐ Export bulletin PDF par élève ou par classe
-

12.2 Priorité P1 — Phase 2

US-06 — Quiz adaptatif (Élève)

En tant qu'élève, je veux passer un quiz dont la difficulté s'adapte à mon niveau.

Critères :

- ☐ Après 3 réponses correctes consécutives → niveau monte
- ☐ Après 3 erreurs consécutives → niveau descend avec rappel de cours
- ☐ Score affiché à la fin avec corrections détaillées

- ☐ XP attribué immédiatement
 - ☐ Historique de tous les quiz accessibles
-

US-07 — Dashboard décrochage (Admin)

En tant qu'admin, je veux voir quels élèves sont à risque de décrochage cette semaine.

Critères :

- ☐ Liste triée par score de risque (haut → bas)
 - ☐ Indicateurs : absences, niveau IA, streak, notes récentes
 - ☐ Alerte email/notification si score > 0.65
 - ☐ Filtres : classe, matière, période
 - ☐ Export CSV de la liste
-

US-08 — Génération quiz automatique (Enseignant)

En tant qu'enseignant, je veux générer un quiz de 10 questions à partir de mon cours en 1 clic.

Critères :

- ☐ Quiz généré < 15 secondes
 - ☐ Questions QCM + Vrai/Faux par défaut
 - ☐ Enseignant peut modifier/supprimer des questions avant publication
 - ☐ Choix de difficulté avant génération
 - ☐ Notification aux élèves de la classe à la publication
-

13. Interfaces UI par Profil

13.1 Pages par rôle

— ADMIN —	
/admin/dashboard décrochage	Tableau de bord : stats globales, alertes
/admin/users	CRUD utilisateurs (tableau + filtres)
/admin/classes	Gestion classes, inscriptions, emploi du temps
/admin/analytics	Rapports par matière, classe, période
/admin/events	Calendrier scolaire
/admin/settings	Configuration école, année scolaire
— ENSEIGNANT —	
/teacher/dashboard	Mes classes du jour, dernières activités
/teacher/courses	Mes cours (liste + upload)
/teacher/courses/:id	Détail cours, fichiers, quiz, statistiques
/teacher/grades	Carnet de notes (par classe/matière)
/teacher/absences	Saisie absences du jour
/teacher/messages	Messagerie (parents/admin)
/teacher/analytics	Performance de mes élèves
— ÉLÈVE —	
/student/dashboard	Aujourd'hui : cours du jour, quiz dispo, XP
/student/courses	Mes cours accessibles
/student/courses/:id	Cours + Mentor IA intégré
/student/quizzes	Quizzes disponibles + historique
/student/grades	Mes notes et bulletins
/student/timetable	Mon emploi du temps
/student/profile	Avatar, XP, badges, classement
— PARENT —	
/parent/dashboard	Vue enfant(s) : dernières notes, absences
/parent/absences	Historique absences + justification
/parent/grades	Bulletin et notes en temps réel
/parent/timetable	Emploi du temps enfant
/parent/messages	Messagerie enseignants/admin
/parent/events	Calendrier examens et événements

13.2 Composants UI critiques

- `<AChat />` — Interface Mentor IA avec streaming SSE, sources citées, feedback
- `<AdaptiveQuizPlayer />` — Player quiz avec timer, feedback immédiat, animation niveau
- `<GradeBook />` — Tableau notes éditables inline, calculs automatiques
- `<AbsenceTracker />` — Calendrier + saisie rapide absences
- `<XPDashboard />` — Avatar évolutif, barre XP, badges, classement
- `<NotificationCenter />` — Cloche avec badge, panneau coulissant

14. Sécurité & Conformité

14.1 Implémentation JWT

```
# core/security.py

ACCESS_TOKEN_EXPIRE_MINUTES = 15
REFRESH_TOKEN_EXPIRE_DAYS = 7

def create_access_token(user_id: str, role: str) -> str:
    payload = {
        "sub": user_id,
        "role": role,
        "type": "access",
        "iat": datetime.utcnow(),
        "exp": datetime.utcnow() + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    }
    return jwt.encode(payload, SECRET_KEY, algorithm=ALGORITHM)

def create_refresh_token(user_id: str) -> tuple[str, str]:
    """Retourne (token_raw, token_hash) – stocker le hash en DB."""
    token_raw = secrets.token_urlsafe(48)
    token_hash = hashlib.sha256(token_raw.encode()).hexdigest()
    return token_raw, token_hash
```

14.2 Chiffrement des messages

```
# core/security.py

from cryptography.hazmat.primitives.ciphers.aead import AESGCM
import os

def encrypt_message(content: str, key: bytes) -> tuple[str, str]:
    """Chiffre un message. Retourne (ciphertext_b64, iv_b64)."""
    iv = os.urandom(12) # 96 bits pour GCM
    aesgcm = AESGCM(key)
    ciphertext = aesgcm.encrypt(iv, content.encode(), None)
    return base64.b64encode(ciphertext).decode(), base64.b64encode(iv).decode()

def decrypt_message(ciphertext_b64: str, iv_b64: str, key: bytes) -> str:
    ciphertext = base64.b64decode(ciphertext_b64)
    iv = base64.b64decode(iv_b64)
    aesgcm = AESGCM(key)
    return aesgcm.decrypt(iv, ciphertext, None).decode()
```

14.3 Middleware de sécurité (à configurer dans Nginx)

```
# nginx/nginx.conf (extrait sécurité)
add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
add_header X-Frame-Options "DENY" always;
add_header X-Content-Type-Options "nosniff" always;
add_header Referrer-Policy "strict-origin-when-cross-origin" always;
add_header Content-Security-Policy "default-src 'self'; script-src 'self'; style-src 'self'
'unsafe-inline'; img-src 'self' data:;" always;
add_header Permissions-Policy "geolocation=(), microphone=(), camera=()" always;

# Rate limiting
limit_req_zone $binary_remote_addr zone=api:10m rate=20r/s;
limit_req_zone $binary_remote_addr zone=login:10m rate=5r/m;

location /api/v1/auth/login {
    limit_req zone=login burst=3 nodelay;
    proxy_pass http://api;
}
```

14.4 Checklist sécurité déploiement

- ☐ Tous les secrets dans `.env` (jamais dans le code)
- ☐ `.env` exclu du dépôt Git (`.gitignore`)
- ☐ PostgreSQL : pas de connexion directe depuis Internet

- ☐ Redis : authentification activée, pas exposé
- ☐ MinIO : buckets privés, URLs présignées uniquement
- ☐ Ollama : pas exposé hors réseau Docker
- ☐ Grafana/Flower : accès réseau interne seulement
- ☐ Nginx : TLS 1.3 uniquement, HTTP → HTTPS redirect
- ☐ Logs sans données personnelles (pseudonymisation)
- ☐ Backup PostgreSQL quotidien + test restore mensuel

15. Contraintes Non-Fonctionnelles (SLA)

Exigence	Valeur cible	Mesure
Disponibilité	99%	Prometheus uptime
Latence API (p95) hors IA	< 400ms	Prometheus histogram
Premier token Mentor IA (p95)	< 3 secondes	Custom metric
Génération quiz (p95)	< 15 secondes	Celery task duration
Upload fichier 50 Mo	< 30 secondes	Client-side timing
Indexation document (p95)	< 60 secondes	Celery task duration
Notification absence	< 5 minutes	Custom metric
Utilisateurs simultanés supportés	500	Locust load test
Stockage cours (an 1)	500 Go	MinIO dashboard
Rétention logs	90 jours	Loki config
Taille max upload	200 Mo	Nginx + FastAPI
Navigateurs supportés	Chrome 110+, Firefox 115+, Safari 16+	—
Mobile	iOS 15+, Android 11+	Expo config
Sauvegarde DB	Quotidienne, 30 jours de rétention	Script cron

16. Roadmap MVP Phasée

Phase 0 — Infrastructure (Semaines 1–2)

Objectif : Tout tourne, rien ne fonctionne encore en front.

- ☐ Docker Compose complet avec tous les services
- ☐ Migrations Alembic : schéma complet
- ☐ Script seed données initiales (1 admin, 3 classes, 10 élèves, 5 profs, 3 parents)
- ☐ CI : lint (ruff, eslint), tests, build
- ☐ Health check endpoint `/health`
- ☐ MinIO : création des buckets au démarrage
- ☐ Ollama : pull des modèles au démarrage via script

Phase 1 — Auth & Admin (Semaines 3–5)

Livable : L'école peut gérer ses utilisateurs et sa structure.

- ☐ Authentification complète (login, refresh, logout, MFA TOTP)
- ☐ CRUD utilisateurs (admin)
- ☐ Gestion classes, inscriptions, matières
- ☐ Emploi du temps (création + consultation)
- ☐ Interface admin React (pages users, classes, timetable)
- ☐ Logs d'audit

Phase 2 — Cours, Notes & Absences (Semaines 6–9)

Livable : Les enseignants travaillent, les parents sont informés.

- ☐ Upload cours + stockage MinIO
- ☐ CRUD évaluations + saisie notes bulk
- ☐ Publication notes + notifications WebSocket
- ☐ Saisie absences + notification parent temps réel
- ☐ Messagerie sécurisée (chiffrée)
- ☐ App mobile React Native : login, notes, absences, notifications push
- ☐ Bulletins PDF générés côté serveur

Phase 3 — Mentor IA & RAG (Semaines 10–13)

Livable : L'IA aide les élèves sur leurs cours.

- ☐ Pipeline Celery : PDF → extraction → chunks → embeddings → pgvector
- ☐ API chat avec streaming SSE
- ☐ Interface AIChat composant (web + mobile)
- ☐ Génération quiz par IA
- ☐ Player quiz (étudiant)
- ☐ Feedback 👍/👎 sur réponses IA
- ☐ Limite tokens journalière par élève

Phase 4 — Adaptatif & Gamification (Semaines 14–17)

Livable : La plateforme apprend et motive.

- ☐ Algorithme adaptatif (score lissé + niveaux)
- ☐ Quiz adaptatif (difficulté dynamique en cours de session)
- ☐ Système XP + transactions
- ☐ Badges (déclencheurs automatiques)
- ☐ Leaderboard classe
- ☐ Avatar évolutif
- ☐ Dashboard décrochage admin (score de risque)
- ☐ Alertes décrochage automatiques

Phase 5 — Finalisation (Semaines 18–20)

Livable : Plateforme production-ready.

- ☐ Tests de charge Locust (500 VU)
- ☐ Optimisations : cache Redis, lazy loading, pagination cursor
- ☐ PWA (manifest, service worker, offline partiel)
- ☐ Support RTL arabe complet
- ☐ Audit WCAG 2.1 AA
- ☐ Documentation API Swagger complète
- ☐ Runbook opérationnel (procédures déploiement, backup, incidents)
- ☐ Formation utilisateurs (guide admin + guide enseignant)

17. Stratégie de Tests

17.1 Backend (pytest)

```
# Structure tests
backend/app/tests/
├─ conftest.py          # fixtures: db, client, users authentifiés
├─ unit/
│   ├─ test_security.py # JWT, bcrypt, chiffrement
│   ├─ test_adaptive.py # algorithme adaptatif
│   └─ test_dropout.py  # algorithme décrochage
├─ integration/
│   ├─ test_auth.py     # endpoints auth complets
│   ├─ test_courses.py  # upload, indexation
│   ├─ test_grades.py   # saisie, publication, calculs
│   ├─ test_absences.py # saisie + notification
│   └─ test_rag.py       # pipeline RAG end-to-end
└─ e2e/
    └─ test_scenarios.py # scénarios métier complets

# Commandes
pytest --cov=app --cov-report=html --cov-fail-under=70
```

17.2 Frontend (Vitest + Playwright)

```
# Tests composants
src/__tests__/
├─ AIChat.test.tsx
├─ GradeBook.test.tsx
├─ QuizPlayer.test.tsx
└─ AdaptiveScore.test.ts

# Tests E2E Playwright
e2e/
├─ auth.spec.ts          # login, MFA, logout
├─ teacher-flow.spec.ts  # upload cours, saisie notes
├─ student-ai.spec.ts    # conversation Mentor IA
└─ parent-notify.spec.ts # réception notification absence
```

17.3 Tests de charge (Locust)

```
# locustfile.py
class StudentUser(HttpUser):
    wait_time = between(2, 5)

    @task(3)
    def view_courses(self): ...

    @task(2)
    def take_quiz(self): ...

    @task(1)
    def chat_with_ai(self): ...

# Commande : locust --users 500 --spawn-rate 50
# Cible : p95 < 400ms hors IA, 0% erreur
```

17.4 Couverture minimale

Layer	Cible
Backend (pytest)	70% couverture
Composants React	60%
E2E scénarios critiques	100% des P0 US
Load test validé	500 VU simultanés

18. Monitoring & Observabilité

18.1 Métriques Prometheus exposées

```
# Dans FastAPI (prometheus-fastapi-instrumentator)
# Auto-généré :
http_request_duration_seconds{method, endpoint, status}
http_requests_total{method, endpoint, status}

# Custom metrics à implémenter :
ai_response_first_token_seconds      # latence streaming IA
ai_sessions_active                   # sessions IA actives
documents_indexed_total               # documents indexés
quiz_attempts_total                  # tentatives quiz
notifications_sent_total{type}       # notifications envoyées
dropout_risk_high_count               # élèves à risque élevé
```

18.2 Alertes Grafana

```
# Alertes à configurer
- name: "API Latence Élevée"
  condition: http_request_duration_seconds p95 > 1s pendant 5 min

- name: "IA Lente"
  condition: ai_response_first_token_seconds p95 > 5s

- name: "Backup Manqué"
  condition: last_backup_timestamp > 26h

- name: "Disque Plein"
  condition: disk_usage > 85%

- name: "Trop d'Élèves à Risque"
  condition: dropout_risk_high_count > 10
```

18.3 Logs structurés (JSON)

```
# Chaque log inclut :
{
  "timestamp": "ISO8601",
  "level": "INFO|WARNING|ERROR",
  "service": "api|worker|beat",
  "request_id": "uuid",          # tracé de bout en bout
  "user_id": "uuid_anonymisé",  # pseudonymisé
  "action": "string",
  "duration_ms": 42,
  "message": "string"
}
```

19. Internationalisation

19.1 Configuration react-i18next

```
// src/i18n/index.ts
import i18n from 'i18next';
import { initReactI18next } from 'react-i18next';

i18n.init({
  resources: {
    fr: { translation: require('./fr.json') },
    ar: { translation: require('./ar.json') }
  },
  lng: 'fr',
  fallbackLng: 'fr',
  interpolation: { escapeValue: false }
});

// Support RTL
document.dir = i18n.language === 'ar' ? 'rtl' : 'ltr';
```

19.2 Classes Tailwind RTL

```
<!-- Utiliser les préfixes rtl: pour support automatique -->
<div class="ml-4 rtl:ml-0 rtl:mr-4">...</div>
<div class="text-left rtl:text-right">...</div>
```

20. KPIs & Métriques de Succès

KPI	Baseline	Mois 3	Mois 6	Mois 12
Taux d'adoption enseignants	0%	60%	80%	95%
Taux d'adoption parents	0%	40%	65%	80%
Sessions Mentor IA / élève / semaine	0	2	4	6+
Temps réponse API p95	—	< 500ms	< 400ms	< 300ms
Précision alertes décrochage	—	—	> 65%	> 75%
Uptime mensuel	—	99%	99%	99.5%
Score satisfaction enseignants (NPS)	—	> 20	> 40	> 60
Réduction taux décrochage détecté	—	—	-10%	-20%

21. Glossaire

Terme	Définition
RAG	Retrieval Augmented Generation — technique IA qui enrichit le prompt avec des documents pertinents
pgvector	Extension PostgreSQL pour stocker et rechercher des vecteurs d'embeddings
Embedding	Représentation vectorielle d'un texte permettant la recherche par similarité
Chunk	Fragment de document (512 tokens) créé lors de l'indexation RAG
TOTP	Time-based One-Time Password — code MFA qui change toutes les 30 secondes
JWT	JSON Web Token — token d'authentification signé
SSE	Server-Sent Events — protocole de streaming unidirectionnel serveur → client
Adaptive Level	Niveau de difficulté calculé dynamiquement : remediation / normal / advanced
Dropout Risk	Score de risque de décrochage scolaire calculé par l'IA (0.0 à 1.0)
XP	Points d'expérience — monnaie de la gamification
MinIO	Solution de stockage objet open-source compatible S3
Celery	Framework Python pour l'exécution de tâches asynchrones en background
pgvector IVFFlat	Index pgvector optimisé pour la recherche approximative de voisins proches

— Fin du PRD Don Bosco Connect v2.0 —

Instruction pour l'agent de développement : Ce document est le seul point de vérité. Implémenter dans l'ordre des phases (0 → 5). Chaque phase doit être entièrement testée avant de passer à la suivante. En cas d'ambiguïté, favoriser la simplicité et la sécurité. Stack non négociable : respecter les versions exactes spécifiées en section 5.